

Reviewer Pack — Minimal Hodge Index of Tropicalized Boolean Functions Bounds...

Entry 629b88289988 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 10:53:53 UTC

Minimal Hodge Index of Tropicalized Boolean Functions Bounds Resolution Proof Width for Random k-CNF

Entry ID: 629b88289988 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 10:53:53 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry × Boolean Function Complexity **Field B** (complexity object): Resolution proof complexity

Statement:

For random k-CNF instances, the minimal Hodge index of their tropicalized polynomial varieties is correlated with the resolution proof width in such a way that for all $n \geq 10$ and $\alpha \in \{0.1, 0.3, 0.5\}$, there exists a constant $C(\alpha)$ such that $|H_{\min}(f_{\text{trop}}) - \log_2(t(f))| \leq C(\alpha)n^\alpha$, where f is the k-CNF formula, f_{trop} is its tropicalized version, and $t(f)$ is the resolution proof width for f .

Rationale (proposer's reasoning):

Tropical geometry provides a bridge between algebraic complexity theory and real-world Boolean functions. The Hodge index of a variety in tropical geometry could potentially reveal structural properties about the function's algebraic structure that are relevant to the resolution proof complexity. This conjecture aims to explore such a connection, which has not been widely studied.

Taxonomy category: tropical_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 77a0977bae39bc71

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all $n \geq 10$ and $\alpha \in \{0.1, 0.3, 0.5\}$, the Spearman rank correlation coefficient between $H_{\min}(f_{\text{trop}})$ and $\log_2(t^*(f))$ exceeds 0.7, with at least 30 seeds and a p-value ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.90	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Hodge index AND Tropical Geometry AND Boolean Function Complexity AND Resolution proof complexity - tropicalization of Boolean functions IN Tropical Geometry AND correlation WITH Resolution proof width - Hodge index OF tropicalized polynomial varieties IN k-CNF AND related TO resolution proof width

Top relevant hits considered: - [http://arxiv.org/abs/2604.21709v1] Residues of a tropical zeta function for convex domains - [http://arxiv.org/abs/1901.05780v2] Hodge filtration, minimal exponent, and local vanishing - [http://arxiv.org/abs/0903.1378v2] Mirror symmetry for P^2 and tropical geometry - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmüller and Siegel spaces - [http://arxiv.org/abs/1001.1554v4] Tropical geometry and correspondence theorems via toric stacks - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/2509.25273v1] Hodge Conjecture via Singular Varieties - [http://arxiv.org/abs/1410.1317v2] Sections of the Hodge bundle over Ekedahl-Oort strata of Shimura varieties of Hodge type

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=160.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import itertools

# Function to generate a random k-CNF formula
def generate_k_cnf(n, m):
    if m > n * (n - 1) / 2:
        raise ValueError("Too many clauses for the given number of variables")

    cnf = []
    literals = list(range(1, n + 1)) + [-i for i in range(1, n + 1)]

    for _ in range(m):
        clause = random.sample(literals, k=3)
        cnf.append(clause)

    return cnf

# Function to compute the resolution proof width
```

```

def resolution_width(cnf):
    clauses = set(tuple(sorted(clause)) for clause in cnf)
    resolvents = set()
    stack = list(clauses)

    while stack:
        clause1 = stack.pop()
        if len(clause1) == 1:
            return len(stack) + 1

        for clause2 in clauses:
            if len(clause1) != len(clause2):
                continue

            common = set(clause1).intersection(set(clause2))
            if len(common) == 1:
                resolvent = tuple(sorted(list(set(clause1) ^ set(clause2))))
                if resolvent not in resolvents:
                    resolvents.add(resolvent)
                    stack.append(resolvent)

    return len(stack)

# Function to compute the Hodge index of a tropicalized polynomial
def hodge_index(cnf):
    # This is a placeholder function. In practice, you would need to implement
    # the actual computation of the Hodge index for the tropicalized polynomial.
    # For simplicity, we'll return a dummy value.
    return random.randint(1, 5)

# Function to run one trial with a given seed
def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [10, 20, 30, 40]
    alpha_values = [0.1, 0.3, 0.5]
    results = []

    for n in n_values:
        for alpha in alpha_values:
            m = int(n * (n - 1) / 2)
            cnf = generate_k_cnf(n, m)
            t_star = resolution_width(cnf)
            H_min = hodge_index(cnf)

            if t_star == 0:
                continue

            results.append((H_min, math.log2(t_star), n))

    if not results:
        return {
            "metric_name": "Spearman rank correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,

```

```

        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

H_min_values = [r[0] for r in results]
log2_t_star_values = [r[1] for r in results]

n_max = max(r[2] for r in results)

# Compute Spearman rank correlation
def rank(x):
    return {v: i + 1 for i, v in enumerate(sorted(set(x), reverse=True))}

ranks_H_min = rank(H_min_values)
ranks_log2_t_star = rank(log2_t_star_values)

n = len(results)
sum_d_ranks_squared = sum((ranks_H_min[H_min] - ranks_log2_t_star[log2_t_star]) ** 2 for H_min, log2_t_star in results)
rho_spearman = 1 - (6 * sum_d_ranks_squared) / (n * (n**2 - 1))

return {
    "metric_name": "Spearman rank correlation",
    "metric_value": rho_spearman,
    "instances_tested": len(results),
    "n_max": n_max,
    "conjecture_holds": rho_spearman > 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next((seed for seed, r in zip(seeds, results) if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
erexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
TRIAL: {'metric_name': 'Spearman rank correlation', 'metric_value': None, 'instances_tested': 0, 'n_m
RESULT: INCONCLUSIVE insufficient_support
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests up to $n = 40$ and uses a placeholder function for computing the Hodge index, which returns random values. This is insufficient to confirm the conjecture as it does not provide evidence beyond small instances and lacks an actual implementation of the Hodge index computation.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code only tested up to $n = 40$ and used a placeholder function for computing the Hodge index, which returns random values. This is insufficient to confirm the conjecture as it does not provide evidence beyond small instances and lacks an actual implementation of the Hodge index computation. | next: Implement a proper computation of the Hodge index and test the conjecture with larger instance sizes and more seeds to provide stronger evidence.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20400
2	preregistration	ollama_remote	glm4:latest	0	0	11892
3	novelty	ollama_remote	glm4:latest	0	0	14404
4	novelty	ollama_remote	glm4:latest	0	0	13994
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17920
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17426
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20239
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19383
9	critic	ollama_remote	glm4:latest	0	0	17524
10	judge	ollama_remote	glm4:latest	0	0	18939

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 172121 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/629b88289988.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/629b88289988.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/629b88289988.tar.gz (if generated)